README

HelixCode Supported-Platforms Manifest

This directory is the single source of truth for which operating systems HelixCode supports, in service of **Constitution §11.4.81 — Cross-Platform-Parity Mandate** (tracker item **HXC-015**).

Files

File	Purpose
`supported_platforms.yaml`	Machine-readable manifest of OS targets, their `uname -s` strings, whether they are host-shell targets, and whether CI/test hardware exists.
`README.md`	This file.

What §11.4.81 requires

For every platform-specific primitive, a multi-platform project must ship a **per-OS equivalent chosen at runtime** via `uname -s` (or equivalent), with an **honest kernel-gap citation** wherever a Linux primitive has no portable equivalent (canonical example: XNU does not enforce `RLIMIT_AS` for unprivileged processes).

How the manifest is consumed

`scripts/gates/cross_platform_parity_gate.sh` (the `CM-CROSS-PLATFORM-PARITY` gate) reads `supported_platforms.yaml`, computes the set of **host-shell platforms** (`host_shell_target: true` → Linux, macOS, Windows), and scans the repo's shell scripts:

- A script that does `case "$(uname -s)"` **dispatch** must cover every host-shell platform's `uname -s` value with a non-SKIP branch **or** carry a documented honest-kernel-gap citation for the missing platform. A script claiming multi-platform dispatch but **silently missing** a manifest platform's branch (no gap citation) is a hard **FAIL**.
- A script that is intentionally Linux-only may declare itself so with a `# PARITY: linux-only — <reason>` marker; the gate then treats it as compliant (this is the pragmatic policy while the phased rename/rewrite is in progress — see the gate header).
- A script that uses platform-specific primitives **without** either a dispatch block or a `# PARITY:` marker is a **soft finding** (reported, non-fatal).

Cross-compiled, non-host-shell targets (iOS, Android, Aurora OS, Harmony OS) carry `host_shell_target: false` and impose **no** `uname -s` branch obligation — they are built *from* a desktop host and have no host shell of their own.

CI/test-hardware honesty

`ci_test_hardware: false` for macOS and Windows is an **honest** statement: no macOS/Windows CI hardware is currently enrolled. Per §11.4.81(B), a platform-dependent test may `SKIP`-with-reason (`hardware_not_present` / `operator_attended`) on those platforms until hardware is enrolled — that is not a bluff, it is a documented gap. Linux is the canonical dev + CI host (`ci_test_hardware: true`).

Sources verified 2026-05-29: project go.mod + CLAUDE.md §3.1 (no third-party operator instructions)

Reviewed against Constitution §11.4.99. This is an internal supported-platforms manifest README: it documents the `supported_platforms.yaml` schema, the `CM-CROSS-PLATFORM-PARITY` gate behaviour, and the §11.4.81 `uname -s` dispatch policy. It contains **no third-party service setup instructions, no external API guidance, and no externally-pinned version numbers** to cross-reference against an online source — the OS targets and their `uname -s` strings (Linux/Darwin/ Windows, plus cross-compiled iOS/Android/Aurora OS/ Harmony OS) live in `supported_platforms.yaml`, not here. Toolchain/version authority for this repo is `go.mod` + CLAUDE.md §3.1 (Go 1.26 / go1.26.3 latest, root 1.25.2). Negative finding: nothing in this file required (or could be) verified against a vendor doc this run.